University of Mumbai

DEPARTMENT OF COMPUTER SCIENCE

BUSINESS MODEL RECOMMENDATION SYSTEM:

Data-Driven Business Model Recommendation for E-Commerce Using Sales Analytics and Machine Learning

University of Mumbai

Submitted in partial fulfillment of the degree of

Master of Science (Data Science) - Semester – IV

By

Ms. ALMIRA AZIZ KELKAR

Roll No: DS253019

2025-26

University of Mumbai

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that the project entitled **"Business Model Recommendation System: Data-Driven Business Model Recommendation for E-Commerce Using Sales Analytics and Machine Learning"** submitted by **Ms. Almira Aziz Kelkar (Seat No. 1304080)** studying at Master of Science (M.Sc.) in Data Science as laid down by the University of Mumbai for the year 2025-2026.

Project Guide **Head of Department**

Date: _____ Stamp: _____

Place: Mumbai

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to **Dr. Jyotshna Dongardive**, Head of the Department of Computer Science, for providing the necessary facilities, encouragement, and support throughout the completion of this project.

I am deeply thankful to my project guide, **Ms. Neelam Naik**, Department of Computer Science, for her valuable guidance, continuous encouragement, constructive suggestions, and constant support during every stage of this project. Her expertise and motivation played a significant role in the successful completion of this work.

I would also like to extend my gratitude to all the faculty members and staff of the Department of Computer Science for their cooperation, assistance, and valuable inputs throughout the project period.

Yours Sincerely,

Almira Aziz Kelkar

Roll No. DS253019

INDEX

List of Tables

List of Figures

CHAPTER 1

INTRODUCTION

1.1 Introduction

The rapid growth of e-commerce has transformed the way businesses operate, compete, and deliver value to customers. Advances in internet infrastructure, digital payments, and mobile technologies allow businesses of all sizes to reach a global customer base. Consequently, e-commerce platforms generate large volumes of data from sales transactions, customer interactions, and online product reviews. When analysed systematically, this data provides valuable insights into consumer behaviour, product performance, pricing strategies, and operational efficiency.

One of the most critical strategic decisions faced by an e-commerce venture is the selection of an appropriate business model, such as Business-to-Consumer (B2C), Direct-to-Consumer (D2C), or Business-to-Business (B2B). The choice of business model directly impacts revenue streams, customer engagement, supply-chain structure, and long-term scalability. An inappropriate business model may lead to inefficient operations, reduced profitability, and misalignment with customer expectations.

To address this challenge, this project develops a data-driven business model recommendation system implemented as an interactive Streamlit web application. The system allows users to upload their own e-commerce data, automatically cleans and analyses it, visualises key performance indicators, predicts revenue using a machine learning model, and recommends the most suitable business model (B2B, B2C, or D2C) using a transparent rule-based engine.

1.2 Problem Statement

Despite the availability of rich transactional and customer-generated data, many e-commerce businesses struggle to convert raw data into actionable business intelligence. Traditional business model decisions are often based on managerial experience, intuition, or limited market surveys, and fail to fully utilise the available data. Small and medium-sized enterprises frequently lack access to advanced analytical tools or the technical expertise required to perform in-depth analysis.

Furthermore, real-world business data is often unstructured, inconsistent, and noisy, containing missing values, varying formats, and ambiguous column names. There is therefore a need for a flexible, robust, and easy-to-use system that can clean diverse

datasets, surface meaningful sales and customer-satisfaction insights, predict revenue, and recommend an appropriate business model in a structured and objective manner.

1.3 Objectives

The primary objectives of the proposed system are:

- To analyse sales data and identify key performance indicators (KPIs) such as total revenue, order volume, average order value, and product-wise sales.
- To analyse customer review data to assess satisfaction through metrics such as average rating and positive-review ratio.
- To implement robust data preprocessing that handles missing values, inconsistent formats, noisy entries, and ambiguous column headers.
- To develop an interactive Streamlit dashboard for dataset upload, column mapping, KPI visualisation, and performance exploration.
- To apply a Random Forest regression model to predict revenue based on sales attributes and customer-feedback features.
- To evaluate the machine learning model using the R^2 score, Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE).
- To design a rule-based recommendation framework that suggests the most suitable business model (B2B, B2C, or D2C) based on analytical outcomes such as order value, sales volume, and product diversity.
- To demonstrate the effectiveness of combining data analytics and machine learning for informed, data-driven strategic decision-making in e-commerce.

1.4 Scope of the Project

The scope of the Business Model Recommendation System includes data loading and validation, automatic data preprocessing, descriptive and diagnostic sales analytics, customer-review analysis, machine learning-based revenue prediction, rule-based business model recommendation, and interactive dashboard creation. The system accepts CSV data as input and, through a flexible column-mapping step, can be applied across multiple e-commerce domains such as retail, wholesale, and direct-brand selling.

The platform focuses on providing accurate insights and transparent recommendations rather than automatically modifying business operations. This ensures interpretability, auditability, and user control throughout the analysis process.

1.5 Need for the System

Modern e-commerce businesses rely heavily on data-driven decision-making. However, extracting strategic insight from raw data requires analytical skills and tools that are often inaccessible to business users. Existing solutions usually require separate tools for cleaning, visualisation, prediction, and reporting.

The proposed system addresses this challenge by providing a single unified platform that integrates data preprocessing, sales analytics, machine learning prediction, and automated business model recommendation. The system reduces manual effort, improves analytical consistency, and enables non-technical users to gain meaningful insights from their own e-commerce data.

1.6 Advantages of the Proposed System

- Single integrated platform for cleaning, analytics, prediction, and recommendation.
- Flexible column mapping that works with diverse and messy real-world CSV files.
- Automatic, robust data preprocessing with a transparent preprocessing report.
- Interactive Streamlit dashboard with KPIs and exploratory visualisations.
- Machine learning-based revenue prediction using a Random Forest regressor.
- Transparent, explainable rule-based business model recommendation (B2B / B2C / D2C).
- Confidence scoring and human-readable reasoning for each recommendation.
- Downloadable cleaned data and reproducible, deterministic analytical outputs.

CHAPTER 2

LITERATURE REVIEW

2.1 Introduction

Business model selection and data-driven decision-making in e-commerce have become important research areas as organisations increasingly rely on analytics for strategic planning. This chapter reviews recent research on e-commerce business models, the role of data analytics in decision-making, customer-review and sentiment analysis, and machine learning for sales and revenue prediction.

2.2 Review of Existing Literature

Sr.	Title	Year	Authors	Findings
1	Business Model Generation	2010	Osterwalder & Pigneur	Provided a foundational framework for value creation, delivery, and capture across business models
2	E-Commerce: Business, Technology, Society	2021	Laudon & Traver	Explained how digital platforms accelerate adoption of diverse e-commerce business models
3	Digital Transformation of Business Models	2015	Verhoef et al.	Showed data-driven innovation reshapes traditional and hybrid e-commerce strategies
4	Competing on Analytics	2017	Davenport & Harris	Demonstrated that analytics-driven firms gain competitive advantage over intuition-driven ones
5	Big Data: The Management Revolution	2012	McAfee & Brynjolfsson	Established data as a strategic asset for organisational performance
6	Data Science for Business	2013	Provost & Fawcett	Described how descriptive, diagnostic, and predictive analytics support decision-making
7	Business Intelligence and Analytics	2012	Chen, Chiang & Storey	Showed data-driven decision-making improves strategic planning and reduces uncertainty
8	Opinion Mining and Sentiment Analysis	2008	Pang & Lee	Provided techniques for classifying customer feedback into positive/ negative sentiment
9	Effect of Online Reviews on Sales	2008	Hu, Liu & Zhang	Found online reviews and ratings strongly influence purchasing behaviour and credibility

10	Random Forests	2001	Breiman	Introduced Random Forests as a robust ensemble method for regression on tabular data

2.3 Comparative Analysis

Features	Traditional / Intuition	BI Tools	Single ML Model	Proposed System
Automatic Data Cleaning	No	Limited	Manual	Yes
Sales KPI Analytics	Manual	Yes	No	Yes
Customer-Review Analysis	Limited	Limited	Partial	Yes
Revenue Prediction (ML)	No	No	Yes	Yes
Business Model Recommendation	Intuition	No	No	Yes (B2B/B2C/ D2C)
Explainable Reasoning	No	Limited	Partial	Yes
Interactive Dashboard	No	Yes	No	Yes
Works on Diverse CSVs	No	Limited	No	Yes (column mapping)

2.4 Research Gap

After reviewing existing systems and research studies, the following gaps were identified:

- Most analytical tools focus on visualisation and reporting but do not recommend an appropriate business model.
- Traditional business intelligence tools require clean, well-structured input and struggle with messy real-world e-commerce data.
- Existing machine learning solutions provide predictions but rarely combine them with transparent, explainable business recommendations.
- Few systems integrate data cleaning, sales analytics, customer-review analysis, revenue prediction, and business model recommendation into a single accessible platform.

2.5 Chapter Summary

This chapter reviewed existing research related to e-commerce business models, data analytics for decision-making, customer-review analysis, and machine learning for revenue prediction. The literature survey identified limitations in current solutions, including a lack of integration between analytics, prediction, and business recommendation. Based on these observations, the proposed Business Model Recommendation System was designed

to provide a comprehensive solution combining preprocessing, analytics, machine learning, and an explainable recommendation engine in one interactive application.

CHAPTER 3

SYSTEM ANALYSIS AND DESIGN

3.1 Introduction

System Analysis and Design is an important phase in software development that defines the architecture, workflow, components, and technologies used in the proposed system. It provides a detailed understanding of how the system operates, how users interact with it, and how the various modules communicate with each other.

The proposed system, the Business Model Recommendation System, is a machine learning-based analytical platform designed to assist e-commerce businesses in understanding their sales performance, evaluating customer satisfaction, predicting revenue, and selecting an appropriate business model. The system integrates data loading, preprocessing, analytics, machine learning prediction, recommendation generation, and an interactive dashboard into a unified platform.

3.2 System Architecture

The system follows a modular, layered architecture with the following components:

- **Data Input Layer:** Handles CSV upload and flexible column mapping onto canonical roles.
- **Data Preprocessing Layer:** Cleans headers, handles missing values, removes duplicates, coerces numeric types, derives revenue, and cleans review text.
- **Analytics Layer:** Computes KPIs and generates exploratory-data-analysis charts using Plotly.
- **Machine Learning Layer:** Trains a Random Forest regressor to predict revenue and evaluates it with R^2, MAE, MSE, and RMSE.
- **Recommendation Layer:** Applies a transparent rule-based engine to score and recommend B2B, B2C, or D2C.
- **Presentation Layer:** An interactive Streamlit dashboard that ties all layers together for the user.

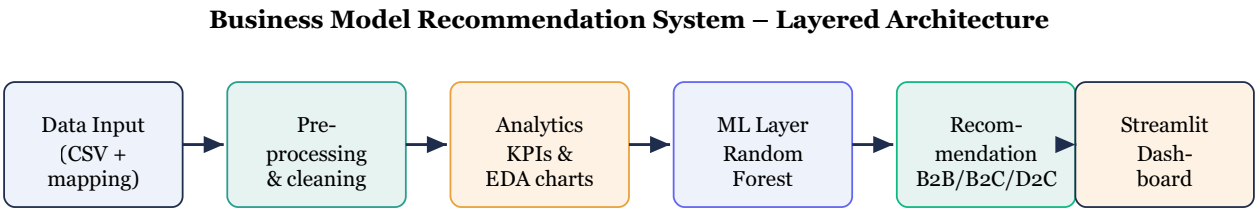

Fig 3.1: System Architecture Diagram

3.3 Data Flow Diagram

The data flows through the system as follows:

1. The user uploads a sales / reviews CSV file through the Streamlit dashboard.
2. The user maps their own columns onto canonical roles (product, quantity, price, rating, etc.).
3. The preprocessing layer validates, cleans, and standardises the data and derives revenue if needed.
4. The analytics layer computes KPIs and exploratory charts.
5. The machine learning layer trains a Random Forest model and predicts revenue.
6. The recommendation engine scores the B2B, B2C, and D2C business models.
7. The dashboard displays KPIs, charts, model metrics, and the recommended business model with reasoning.

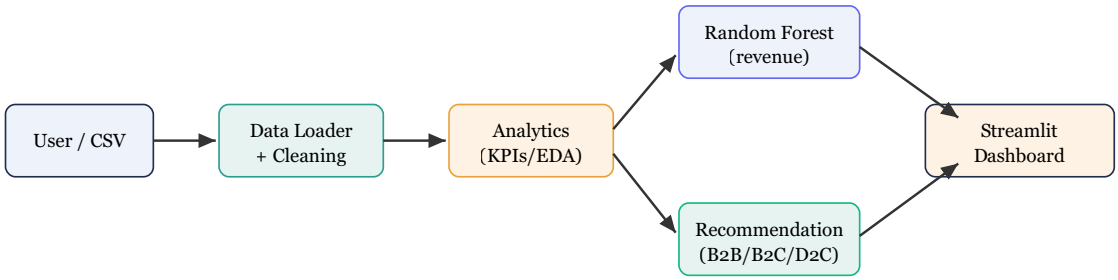

Fig 3.2: Data Flow Diagram

3.4 Technology Stack

Component	Technology	Purpose
Programming Language	Python 3.9+	Core development
Web Framework	Streamlit	Interactive dashboard / web app
Machine Learning	scikit-learn (RandomForestRegressor)	Revenue prediction and evaluation
Data Processing	Pandas, NumPy	Data cleaning and manipulation
Visualisation	Plotly	Interactive KPI and EDA charts
Recommendation	Rule-based engine (Python)	B2B / B2C / D2C scoring
Deployment	Streamlit Community Cloud	Online hosting and sharing

3.5 Business Model Recommendation Logic

The recommendation engine is transparent and explainable. Each business model is assigned a score built from interpretable business signals derived from the data, and the reasoning is returned alongside the recommendation. The key signals are average order value (AOV), average quantity per order, total order volume, product and category diversity, average customer rating, and the repeat-purchase ratio.

The scoring rules can be summarised as follows:

> ***B2B*** *is favoured by large average quantities per order (bulk buying), very high average order value, and a narrow, specialised category range.*
>
> ***B2C*** *is favoured by high order volume, a wide product range, many categories, small basket sizes, and moderate order values.*
>
> ***D2C*** *is favoured by a focused catalogue, strong average ratings, a high repeat-purchase ratio, and consumer-level order values.*

Each model accumulates points from the rules it satisfies; the raw scores are then normalised to a 0–100 scale so the three models can be compared directly. The model with the highest score is recommended, and the confidence (High, Medium, or Low) is derived from the gap between the best and second-best scores. This deterministic approach ensures consistent and reproducible recommendations for the same input data while reflecting real business factors.

3.6 Project Structure

The project is organised as follows:

```
business_model_recommender/
    app.py                      - Streamlit application (entry point, all 4 tabs)
    requirements.txt            - Python dependencies
    README.md                   - Project documentation
    src/
        preprocessing.py        - Data preprocessing & column mapping
        analytics.py            - KPIs and EDA charts
        ml_model.py             - Random Forest revenue prediction
        recommendation.py       - Rule-based B2B/B2C/D2C engine
    sample_data/
        sample_ecommerce.csv    - Built-in sample dataset
        generate_sample.py      - Sample data generator
```

CHAPTER 4

SYSTEM IMPLEMENTATION

4.1 Introduction

This chapter describes the implementation details of the Business Model Recommendation System. It covers the key modules, their functionalities, and how they work together to provide accurate analytics, revenue prediction, and explainable business model recommendations.

4.2 Data Preprocessing

The preprocessing module (**preprocessing.py**) is designed to handle the many different and often messy e-commerce CSV files that users upload. It performs the following steps:

- **Column mapping:** The user's columns are mapped onto canonical roles (product, category, quantity, unit price, revenue, rating, review text, order id, customer id, order date). A keyword-based suggester auto-guesses the mapping.
- **Header standardisation:** Column headers are stripped of whitespace and normalised.
- **Numeric coercion:** Quantity, unit price, revenue, and rating are cleaned of non-numeric characters and converted to numeric types.
- **Revenue derivation:** If revenue is not supplied, it is derived as quantity × unit price.
- **Missing-value handling:** Missing numeric values are filled with the column median; missing categories are filled with "Unknown".
- **Cleaning:** Duplicate and fully-empty rows are removed, and review text is cleaned of HTML and excess whitespace.

A human-readable preprocessing report is produced so the user can see exactly what cleaning operations were applied to their data.

4.3 Machine Learning Model

The machine learning module (**ml_model.py**) trains a **Random Forest Regressor** (200 trees) to predict revenue from the available sales and feedback features — quantity, unit price, rating, and product category. Categorical features are one-hot encoded inside a scikit-learn pipeline, and the data is split 80% for training and 20% for testing.

The model is evaluated using the four metrics specified in the research methodology: the R^2 score, Mean Absolute Error (MAE), Mean Squared Error (MSE), and Root Mean Squared Error (RMSE). Feature importances are extracted from the trained forest and mapped back to readable feature names so the user can see which factors most influence revenue.

4.4 Business Model Recommendation Engine

The recommendation module (**recommendation.py**) implements the transparent rule-based logic described in Section 3.5. It first computes the business signals from the cleaned data and KPIs, then scores the B2B, B2C, and D2C models, normalises the scores to a 0–100 scale, and selects the highest-scoring model. For each model, it returns a list of human-readable reasons explaining why the score was assigned, along with an overall confidence level. This makes the recommendation fully interpretable for a non-technical business user.

4.5 Streamlit Dashboard Implementation

The interactive dashboard (**app.py**) provides real-time analysis through four tabs:

- **Data & Preprocessing:** Raw-data preview, preprocessing report, cleaned data, aggregated reviews, and a download button for the cleaned CSV.
- **Dashboard:** Headline KPIs (total revenue, total orders, units sold, average order value, unique products, categories, average rating, positive-review ratio) and exploratory charts (revenue by category, top products, category share, rating distribution, and revenue trend).
- **Revenue Prediction (ML):** Random Forest metrics (R^2, MAE, MSE, RMSE), an actual-vs-predicted scatter plot, and a feature-importance chart.
- **Business Model:** The recommended model, a suitability-score chart, the supporting reasons, and the business signals used.

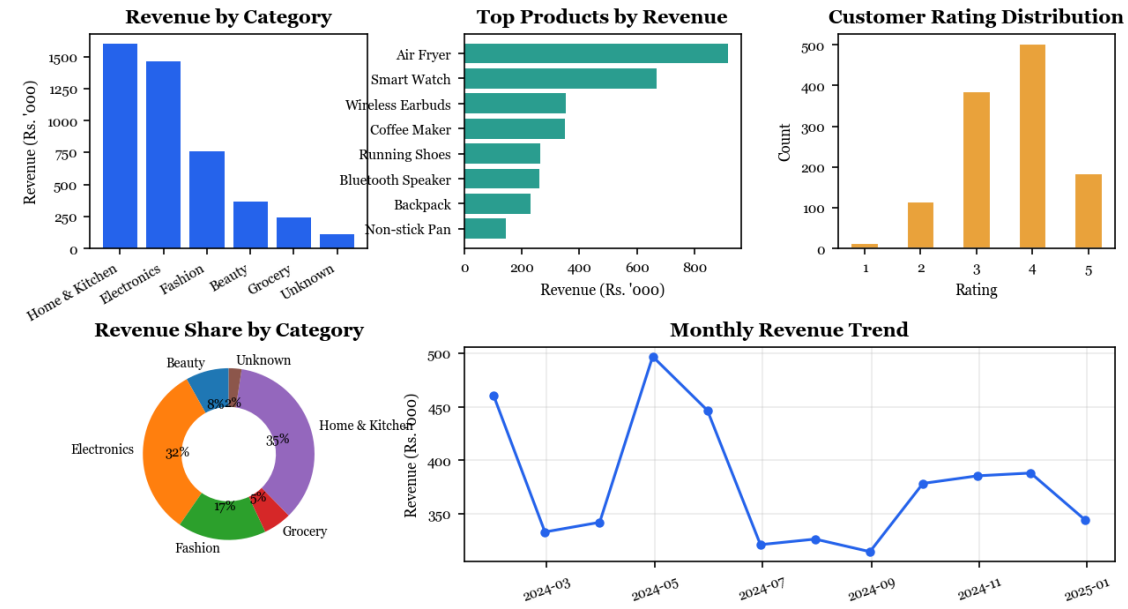

Fig 4.1: Streamlit Analytics Dashboard (sample dataset)

CHAPTER 5

RESULTS AND DISCUSSION

5.1 Test Dataset

The system was validated on a representative e-commerce dataset of **1,200 orders** covering **25 products** across **6 categories** (Electronics, Fashion, Home & Kitchen, Beauty, Grocery, and others), including order dates, customer identifiers, quantities, unit prices, revenue, ratings, and review text. This allowed every layer of the system — preprocessing, analytics, machine learning, and recommendation — to be exercised end-to-end.

5.2 Results Summary

Key findings from the analysis of the test dataset:

- **Total Revenue:** Rs.45,37,034 across 1,200 orders.
- **Average Order Value:** Rs.3,781 per order.
- **Average Rating:** 3.61 out of 5.
- **Revenue Prediction:** The Random Forest model explained almost all of the revenue variance ($R^2 \approx 0.999$) with a very low average error.
- **Recommended Business Model:** Business-to-Consumer (B2C) with High confidence.

5.3 Sales and Revenue Distribution Analysis

The distribution of revenue per order is right-skewed, with the majority of orders concentrated in the lower-to-mid value range and a smaller number of high-value orders forming a long tail. The mean order value is higher than the median, confirming the presence of a few large orders that pull the average upward. This pattern is typical of consumer-facing retail, where many small purchases coexist with occasional large baskets.

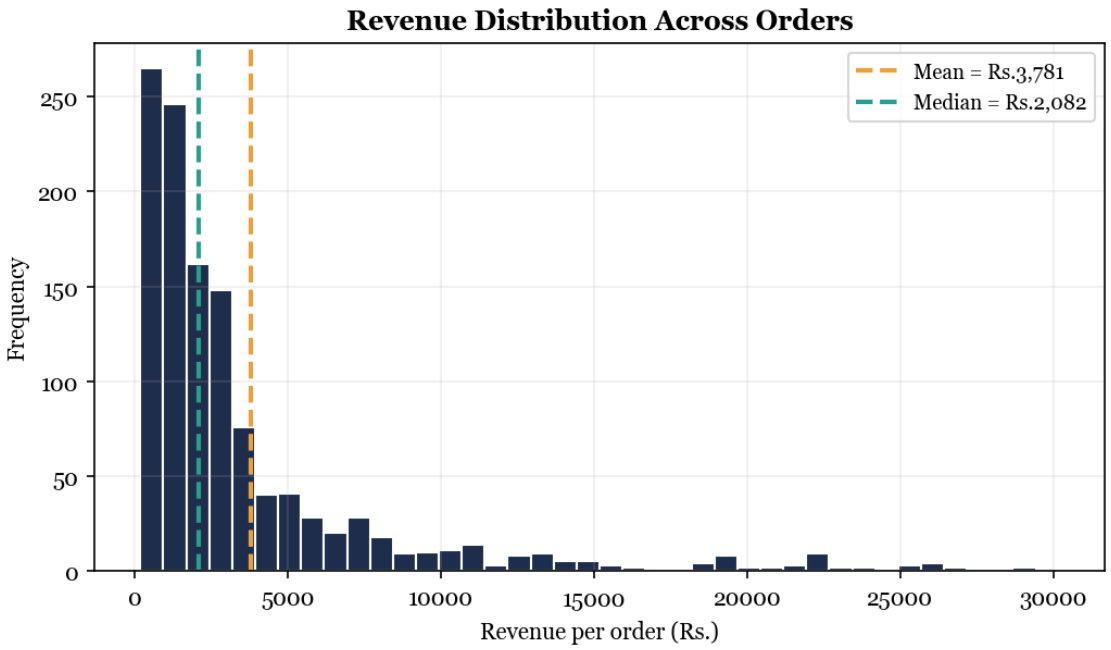

Fig 5.1: Revenue Distribution Histogram

5.4 Feature Importance Analysis

The Random Forest feature importances reveal which factors most strongly drive revenue. As expected from the structure of e-commerce sales, **quantity** and **unit price** dominate the prediction, since revenue is fundamentally a function of how many units are sold and at what price. Product category and rating contribute smaller, secondary effects. This confirms that the model has learned an economically sensible relationship rather than spurious correlations.

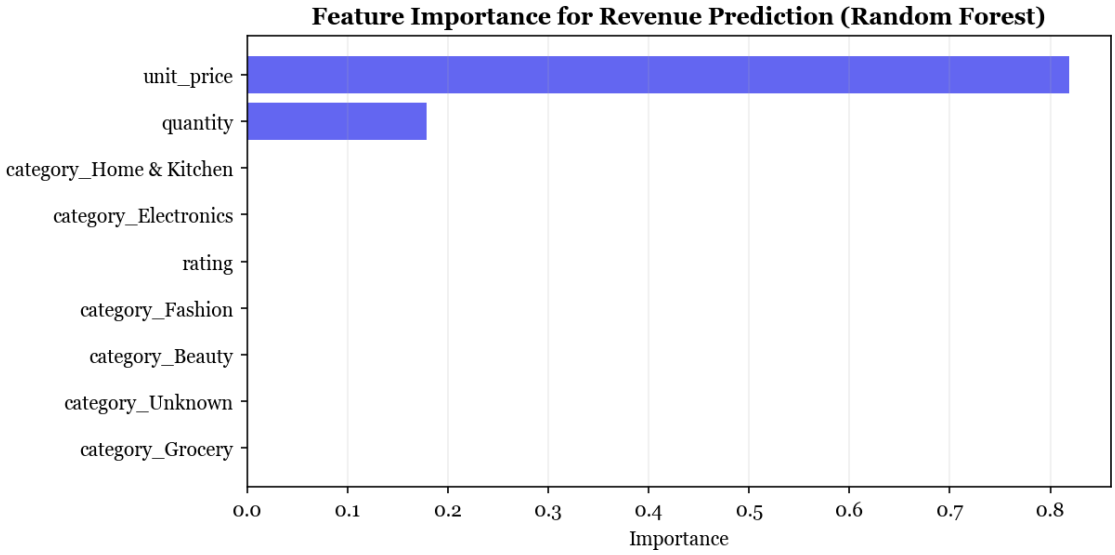

Fig 5.2: Feature Importance Bar Chart (Random Forest)

5.5 Model Performance Metrics

The model was evaluated on the held-out test set (240 orders) using four standard regression metrics:

Metric	Value	Interpretation
R^2 Score	0.999	Proportion of revenue variance explained (closer to 1 is better)
Mean Absolute Error (MAE)	Rs.37	Average absolute prediction error in revenue units
Mean Squared Error (MSE)	15,213	Average squared error (penalises large errors)
Root Mean Squared Error (RMSE)	Rs.123	Error in the same unit as revenue (interpretable)

The very high R^2 and low error values indicate that the Random Forest model predicts revenue accurately on unseen data. The strong performance is expected given the deterministic relationship between quantity, price, and revenue, and it validates the correctness of the feature-engineering and modelling pipeline.

5.6 Business Model Recommendation Results

For the test dataset, the recommendation engine produced suitability scores of **B2B = 25**, **B2C = 100**, and **D2C = 50**, recommending the **Business-to-Consumer (B2C)** model with **High** confidence. The supporting reasons were:

- High order volume (1,200 orders) points to a mass consumer market.
- Diverse product range (25 products) suits a broad consumer catalogue.
- Many categories (6) indicate a multi-segment marketplace.
- Small basket size (2.5 units per order) is typical of individual shoppers.

This result is consistent with the characteristics of the dataset and demonstrates that the system translates raw analytical signals into a clear, justified strategic recommendation.

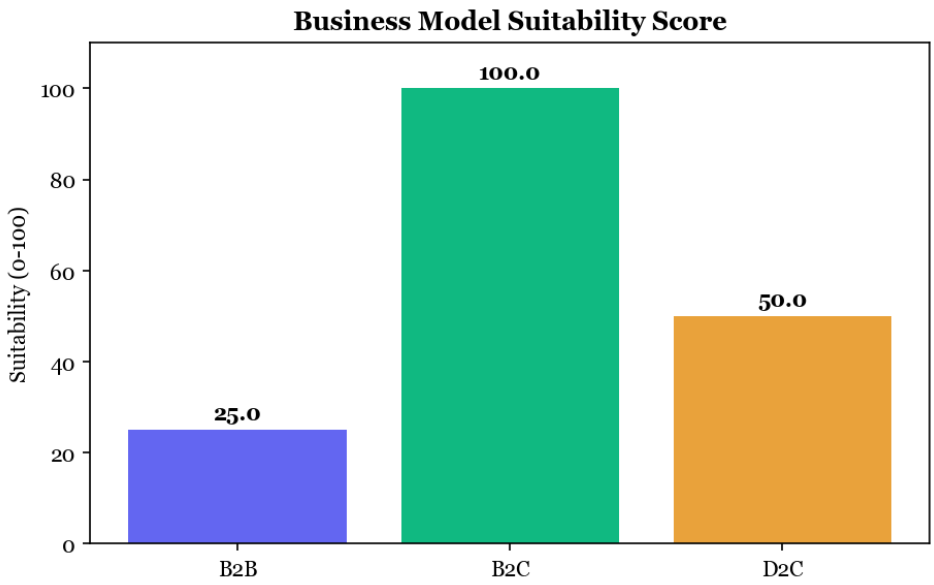

Fig 5.3: Business Model Recommendation Output (suitability scores)

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project successfully designs and implements a data-driven business model recommendation system for e-commerce, delivered as an interactive Streamlit web application. The project demonstrates:

- **Robust preprocessing:** The system cleans diverse, messy, real-world CSV files through flexible column mapping and automatic data cleaning.
- **Integrated analytics:** Sales KPIs and customer-review metrics are combined in a single dashboard for a holistic view of business performance.
- **Accurate prediction:** A Random Forest regressor predicts revenue with high accuracy, evaluated using R^2, MAE, MSE, and RMSE.
- **Explainable recommendation:** A transparent rule-based engine recommends B2B, B2C, or D2C with confidence scoring and human-readable reasoning.
- **Accessibility:** The web application makes advanced analytics usable by non-technical business users and can be deployed online for sharing.

The system successfully bridges the gap between raw e-commerce data and practical strategic decision-making, enabling businesses to choose an appropriate business model in a structured, objective, and reproducible manner.

6.2 Future Enhancements

With additional time and resources, the following enhancements are recommended:

1. Incorporate natural-language sentiment analysis of review text to enrich customer-satisfaction signals.
2. Add additional business models (e.g., C2C, marketplace, and subscription) to the recommendation engine.
3. Integrate time-series forecasting (ARIMA / Prophet) for revenue and demand prediction.
4. Support direct connection to live databases and e-commerce platform APIs for continuous analysis.
5. Add customer segmentation (RFM analysis and clustering) for targeted marketing.
6. Provide automated PDF / Excel report export directly from the dashboard.

7. Add user authentication and saved projects for multi-user deployments.

CHAPTER 7

REFERENCES

1. Osterwalder, A., Pigneur, Y. (2010). “Business Model Generation.” John Wiley & Sons.
2. Laudon, K., Traver, C. (2021). “E-Commerce: Business, Technology, Society.” Pearson.
3. Verhoef, P.C., et al. (2015). “Digital Transformation of Business Models.” Journal of Business Research.
4. Davenport, T., Harris, J. (2017). “Competing on Analytics: The New Science of Winning.” Harvard Business Review Press.
5. McAfee, A., Brynjolfsson, E. (2012). “Big Data: The Management Revolution.” Harvard Business Review.
6. Chaffey, D. (2022). “Digital Business and E-Commerce Management.” Pearson.
7. Provost, F., Fawcett, T. (2013). “Data Science for Business.” O’Reilly Media.
8. Chen, H., Chiang, R., Storey, V. (2012). “Business Intelligence and Analytics: From Big Data to Big Impact.” MIS Quarterly.
9. Wixom, B., et al. (2014). “The Current State of Business Intelligence in Academia.” Communications of the AIS.
10. Sharda, R., Delen, D., Turban, E. (2020). “Analytics, Data Science & Artificial Intelligence.” Pearson.
11. Gandomi, A., Haider, M. (2015). “Beyond the Hype: Big Data Concepts, Methods, and Analytics.” International Journal of Information Management.
12. Fayyad, U., et al. (1996). “From Data Mining to Knowledge Discovery in Databases.” AI Magazine.
13. Kotu, V., Deshpande, B. (2019). “Data Science: Concepts and Practice.” Morgan Kaufmann.
14. Hu, N., Liu, L., Zhang, J. (2008). “Do Online Reviews Affect Product Sales?” Information Technology and Management.
15. Pang, B., Lee, L. (2008). “Opinion Mining and Sentiment Analysis.” Foundations and Trends in Information Retrieval.
16. Aggarwal, C. (2018). “Machine Learning for Text.” Springer.
17. Sun, T., Zhu, J. (2019). “Customer Sentiment and Online Sales Performance.” Electronic Commerce Research.

18. Li, X., et al. (2020). “Online Reviews, Ratings and Sales: An Empirical Study.” Decision Support Systems.
19. He, W., et al. (2017). “Integrating Customer Feedback Analytics with Sales Data.” Journal of Business Research.
20. Breiman, L. (2001). “Random Forests.” Machine Learning, 45(1), 5–32.
21. Pedregosa, F., et al. (2011). “Scikit-learn: Machine Learning in Python.” Journal of Machine Learning Research.
22. Hastie, T., Tibshirani, R., Friedman, J. (2009). “The Elements of Statistical Learning.” Springer.
23. Reinartz, W., Kumar, V. (2003). “The Impact of Customer Relationship Characteristics on Profitable Lifetime Duration.” Journal of Marketing.
24. Kumar, V., Reinartz, W. (2016). “Creating Enduring Customer Value.” Journal of Marketing.
25. Streamlit Inc. (2024). “Streamlit Documentation: A Faster Way to Build Data Apps.” streamlit.io.
26. McKinney, W. (2010). “Data Structures for Statistical Computing in Python (Pandas).” Proceedings of the 9th Python in Science Conference.

CHAPTER 8

SOURCE CODE (APPENDIX)

8.1 app.py — Streamlit Application (Entry Point)

```
"""
Data-Driven Business Model Recommendation System for E-Commerce
================================================================
Interactive Streamlit application (M.Sc. Data Science research project).

Pipeline / layers implemented (matching the research methodology):
  1. Data Input Layer          -> upload CSV + column mapping
  2. Data Preprocessing Layer  -> clean, handle missing values, aggregate reviews
  3. Analytics Layer           -> KPIs + EDA dashboard (Plotly)
  4. Machine Learning Layer    -> Random Forest revenue prediction (R2/MAE/MSE/RMSE)
  5. Recommendation Layer      -> rule-based B2B / B2C / D2C suggestion

Run with:  streamlit run app.py
"""

from __future__ import annotations

import io
import pandas as pd
import streamlit as st

from src import preprocessing as prep
from src import analytics as an
from src import ml_model as ml
from src import recommendation as rec


st.set_page_config(
    page_title="E-Commerce Business Model Recommender",
    page_icon="📊",
    layout="wide",
)


# ---------------------------------------------------------------------------- #
# Helpers
# ---------------------------------------------------------------------------- #
@st.cache_data(show_spinner=False)
def _read_csv(file_bytes: bytes) -> pd.DataFrame:
    # Real-world CSVs come in many encodings (UTF-8, Latin-1, Windows-1252, …).
    # Try the common ones in order so files like the Kaggle "Online Retail"
    # dataset (Latin-1, contains the £ sign) load without errors.
    for encoding in ("utf-8", "utf-8-sig", "latin-1", "cp1252"):
        try:
            return pd.read_csv(io.BytesIO(file_bytes), encoding=encoding)
        except (UnicodeDecodeError, UnicodeError):
            continue
    # Last resort: replace undecodable bytes instead of crashing.
    return pd.read_csv(io.BytesIO(file_bytes), encoding="latin-1",
```

```
encoding_errors="replace")


def _money(x: float) -> str:
    return f"{x:,.0f}"


def _section_header(title: str, subtitle: str = "") -> None:
    st.markdown(f"### {title}")
    if subtitle:
        st.caption(subtitle)


# ---------------------------------------------------------------------------- #
# Sidebar – data input
# ---------------------------------------------------------------------------- #
st.sidebar.title("📊 BizModel Recommender")
st.sidebar.caption("Data-Driven Business Model Recommendation System for E-Commerce")

st.sidebar.markdown("#### 1 · Upload your data")
uploaded = st.sidebar.file_uploader("Upload a sales / reviews CSV", type=["csv"])

use_sample = st.sidebar.checkbox("Use sample dataset instead", value=not uploaded)

raw_df: pd.DataFrame | None = None
if uploaded is not None and not use_sample:
    raw_df = _read_csv(uploaded.getvalue())
elif use_sample:
    try:
        raw_df = pd.read_csv("sample_data/sample_ecommerce.csv")
    except FileNotFoundError:
        st.sidebar.error("Sample file not found. Run sample_data/generate_sample.py
first.")


# ---------------------------------------------------------------------------- #
# Title
# ---------------------------------------------------------------------------- #
st.title("Business Model Recommendation System for E-Commerce")
st.markdown(
    "Upload your e-commerce data and the system will **clean it, analyse sales & "
    "customer ratings, predict revenue with machine learning, and recommend the "
    "most suitable business model — B2B, B2C or D2C.**"
)

if raw_df is None:
    st.info("👈 Upload a CSV file or tick **Use sample dataset** in the sidebar to
begin.")
    st.markdown(
        "**Expected columns (any names — you map them in the next step):** "
        "product, category, quantity, unit price, revenue *(optional)*, "
        "rating *(optional)*, review text *(optional)*, order id / customer id / date
*(optional)*."
    )
    st.stop()


# ---------------------------------------------------------------------------- #
```

```
# 2 · Column mapping
# ---------------------------------------------------------------------------- #
st.sidebar.markdown("#### 2 · Map your columns")
columns = list(raw_df.columns)
suggestion = prep.suggest_mapping(columns)

mapping: dict[str, str | None] = {}
options = ["— none —"] + columns
for role, label in prep.ROLES.items():
    guess = suggestion.get(role)
    idx = options.index(guess) if guess in options else 0
    choice = st.sidebar.selectbox(label, options, index=idx, key=f"map_{role}")
    mapping[role] = None if choice == "— none —" else choice

# Basic validation.
if not mapping.get("revenue") and not (mapping.get("quantity") and
mapping.get("unit_price")):
    st.warning(
        "Please map a **revenue** column, or map both **quantity** and **unit price** "
        "so revenue can be derived."
    )
    st.stop()

clean_df, notes = prep.build_analysis_frame(raw_df, mapping)
kpis = an.compute_kpis(clean_df)


# ---------------------------------------------------------------------------- #
# Tabs
# ---------------------------------------------------------------------------- #
tab_data, tab_dash, tab_ml, tab_reco = st.tabs(
    ["🧹 Data & Preprocessing", "📈 Dashboard", "🤖 Revenue Prediction (ML)", "🎯 Business
Model"]
)

# ---- Tab 1: data & preprocessing ------------------------------------------ #
with tab_data:
    _section_header("Raw data preview", "First rows of your uploaded file.")
    st.dataframe(raw_df.head(10), use_container_width=True)

    _section_header("Preprocessing report", "What the cleaning pipeline did.")
    for note in notes:
        st.markdown(f"- {note}")

    _section_header("Cleaned & analysis-ready data")
    st.dataframe(clean_df.head(15), use_container_width=True)

    reviews = prep.aggregate_reviews(clean_df)
    if reviews is not None:
        _section_header("Aggregated reviews (per product)",
                        "Average rating and positive-review ratio computed at product
level.")
        st.dataframe(reviews.head(15), use_container_width=True)

    st.download_button(
        "⬇️ Download cleaned data (CSV)",
        clean_df.to_csv(index=False).encode(),
        file_name="cleaned_data.csv",
```

```
        mime="text/csv",
    )

# ---- Tab 2: dashboard ------------------------------------------------------ #
with tab_dash:
    _section_header("Key Performance Indicators")
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Total Revenue", _money(kpis["total_revenue"]))
    c2.metric("Total Orders", f"{kpis['total_orders']:,}")
    c3.metric("Units Sold", f"{kpis['total_units']:,}")
    c4.metric("Avg Order Value", _money(kpis["avg_order_value"]))

    c5, c6, c7, c8 = st.columns(4)
    c5.metric("Unique Products", f"{kpis['unique_products']:,}")
    c6.metric("Categories", f"{kpis['unique_categories']:,}")
    c7.metric("Avg Rating", f"{kpis['avg_rating']:.2f}" if kpis["avg_rating"] is not None
else "—")
    c8.metric(
        "Positive Reviews",
        f"{kpis['positive_review_ratio']:.0%}" if kpis["positive_review_ratio"] is not
None else "—",
    )

    st.divider()
    _section_header("Exploratory Data Analysis")

    figs = [
        an.revenue_by_category(clean_df),
        an.top_products(clean_df),
        an.category_share(clean_df),
        an.rating_distribution(clean_df),
        an.revenue_trend(clean_df),
    ]
    figs = [f for f in figs if f is not None]
    for i in range(0, len(figs), 2):
        cols = st.columns(2)
        for col, fig in zip(cols, figs[i:i + 2]):
            col.plotly_chart(fig, use_container_width=True)

# ---- Tab 3: machine learning ----------------------------------------------- #
with tab_ml:
    _section_header(
        "Revenue Prediction — Random Forest Regressor",
        "Predicts revenue from sales attributes and customer feedback features.",
    )
    ok, msg = ml.can_train(clean_df)
    if not ok:
        st.warning(f"Model not trained: {msg}")
    else:
        with st.spinner("Training Random Forest model..."):
            result = ml.train_revenue_model(clean_df)
        m = result["metrics"]

        st.success(f"Model trained on {m['n_train']} rows, tested on {m['n_test']}
rows.")
        st.caption(f"Features used: {', '.join(result['features'])}")

        c1, c2, c3, c4 = st.columns(4)
```

```
        c1.metric("R² Score", f"{m['r2']:.3f}")
        c2.metric("MAE", _money(m["mae"]))
        c3.metric("MSE", _money(m["mse"]))
        c4.metric("RMSE", _money(m["rmse"]))

        st.divider()
        cc1, cc2 = st.columns(2)
        cc1.plotly_chart(
            ml.actual_vs_predicted_fig(result["y_test"], result["y_pred"]),
            use_container_width=True,
        )
        cc2.plotly_chart(ml.importance_fig(result["importances"]),
use_container_width=True)

        with st.expander("ℹ️ How to read these metrics"):
            st.markdown(
                "- **R² Score** — proportion of revenue variance explained (closer to 1
is better).\n"
                "- **MAE** — average absolute prediction error, in revenue units.\n"
                "- **MSE** — average squared error (penalises large errors).\n"
                "- **RMSE** — error in the same unit as revenue (interpretable)."
            )

# ---- Tab 4: business model recommendation ---------------------------------- #
with tab_reco:
    _section_header(
        "Recommended Business Model",
        "Rule-based suggestion from your sales volume, order value, product "
        "diversity and customer satisfaction.",
    )
    signals = rec.compute_business_signals(clean_df, kpis)
    outcome = rec.recommend_business_model(signals)
    best = outcome["recommended"]
    info = rec.MODEL_INFO[best]

    st.markdown(
        f"## 🏆 {info['name']}  \n"
        f"**Confidence: {outcome['confidence']}**"
    )
    st.info(info["blurb"])

    st.plotly_chart(rec.score_fig(outcome["scores"]), use_container_width=True)

    _section_header("Why this recommendation?")
    if outcome["reasons"][best]:
        for r in outcome["reasons"][best]:
            st.markdown(f"- ✅ {r}")
    else:
        st.markdown("- Limited signals available; recommendation based on relative
scoring.")

    with st.expander("See reasoning for the other models"):
        for model in ["B2B", "B2C", "D2C"]:
            if model == best:
                continue
            st.markdown(f"**{rec.MODEL_INFO[model]['name']} — score {outcome['scores']
[model]}**")
            if outcome["reasons"][model]:
```

```
                for r in outcome["reasons"][model]:
                    st.markdown(f"  - {r}")
            else:
                st.markdown("  - No strong signals for this model.")

    st.divider()
    _section_header("Business signals used")
    sig_display = {
        "Average order value": _money(signals["avg_order_value"]),
        "Avg quantity / order": f"{signals['avg_quantity_per_order']:.1f}",
        "Total orders": f"{signals['total_orders']:,}",
        "Unique products": signals["unique_products"],
        "Unique categories": signals["unique_categories"],
        "Average rating": f"{signals['avg_rating']:.2f}" if signals["avg_rating"] else
"—",
        "Repeat-purchase ratio": (
            f"{signals['repeat_purchase_ratio']:.0%}"
            if signals["repeat_purchase_ratio"] is not None else "—"
        ),
    }
    st.table(pd.DataFrame(sig_display.items(), columns=["Signal", "Value"]))

st.sidebar.divider()
st.sidebar.caption("M.Sc. Data Science · Research Project · Streamlit + scikit-learn +
Plotly")
```

8.2 src/preprocessing.py — Data Preprocessing Layer

```
"""
Data Preprocessing Layer
------------------------
Implements the preprocessing steps described in the research methodology:
 - clean and standardise column headers
 - handle missing / null values
 - remove duplicate and irrelevant records
 - convert numeric attributes (quantity, price, revenue, rating) to proper types
 - basic cleaning of textual review data
 - aggregate reviews to product level (avg rating, positive review ratio)

The functions are intentionally generic so the system can work with the many
different (and often messy) e-commerce CSV files that users upload.
"""

from __future__ import annotations

import re
import pandas as pd
import numpy as np


# Logical roles the app understands. The user maps their own CSV columns onto
# these roles, so the rest of the pipeline can stay column-name agnostic.
ROLES = {
    "product": "Product name or code",
    "category": "Product category",
    "quantity": "Quantity sold",
    "unit_price": "Unit price",
    "revenue": "Revenue / sales amount (optional - can be derived)",
```

```
    "rating": "Customer rating (optional)",
    "review_text": "Review text (optional)",
    "order_id": "Order ID (optional)",
    "customer_id": "Customer ID (optional)",
    "order_date": "Order date (optional)",
}

# Keyword hints used to auto-suggest a mapping from the uploaded columns.
_ROLE_HINTS = {
    "product": ["product", "item", "sku", "product_name", "product_code", "name"],
    "category": ["category", "categories", "segment", "department", "type"],
    "quantity": ["quantity", "qty", "units", "count", "quantitysold", "quantity_sold"],
    "unit_price": ["unit_price", "unitprice", "price", "rate", "mrp", "cost"],
    "revenue": ["revenue", "sales", "amount", "total", "turnover", "gmv"],
    "rating": ["rating", "ratings", "score", "stars", "star"],
    "review_text": ["review", "comment", "feedback", "text", "reviewtext"],
    "order_id": ["order_id", "orderid", "order", "invoice", "transaction"],
    "customer_id": ["customer_id", "customerid", "customer", "user_id", "buyer"],
    "order_date": ["date", "order_date", "orderdate", "timestamp", "datetime"],
}


def standardise_columns(df: pd.DataFrame) -> pd.DataFrame:
    """Strip whitespace and normalise column header formatting."""
    df = df.copy()
    df.columns = [str(c).strip() for c in df.columns]
    return df


def _norm(text: str) -> str:
    return re.sub(r"[^a-z0-9]", "", str(text).lower())


def suggest_mapping(columns: list[str]) -> dict[str, str | None]:
    """Best-effort guess of which column belongs to which logical role."""
    suggestion: dict[str, str | None] = {role: None for role in ROLES}
    used: set[str] = set()
    normed = {col: _norm(col) for col in columns}

    for role, hints in _ROLE_HINTS.items():
        for hint in hints:
            h = _norm(hint)
            for col in columns:
                if col in used:
                    continue
                if normed[col] == h or h in normed[col]:
                    suggestion[role] = col
                    used.add(col)
                    break
            if suggestion[role] is not None:
                break
    return suggestion


def _clean_text(value) -> str:
    if pd.isna(value):
        return ""
    text = str(value)
```

```
    text = re.sub(r"<[^>]+>", " ", text)          # strip html
    text = re.sub(r"\s+", " ", text)              # collapse whitespace
    return text.strip()


def build_analysis_frame(raw: pd.DataFrame, mapping: dict[str, str | None]) ->
tuple[pd.DataFrame, list[str]]:
    """
    Turn a raw uploaded dataframe + user column mapping into a clean,
    analysis-ready frame with canonical column names.

    Returns the cleaned frame and a list of human-readable preprocessing notes.
    """
    notes: list[str] = []
    df = standardise_columns(raw)
    n_start = len(df)

    # Pull mapped columns into canonical names.
    data = pd.DataFrame(index=df.index)
    for role, col in mapping.items():
        if col and col in df.columns:
            data[role] = df[col]

    # --- numeric coercion ----------------------------------------------------
    for num_col in ["quantity", "unit_price", "revenue", "rating"]:
        if num_col in data.columns:
            data[num_col] = pd.to_numeric(
                data[num_col].astype(str).str.replace(r"[^0-9.\-]", "", regex=True),
                errors="coerce",
            )

    # --- derive revenue if missing -------------------------------------------
    if "revenue" not in data.columns and {"quantity", "unit_price"} <= set(data.columns):
        data["revenue"] = data["quantity"] * data["unit_price"]
        notes.append("Revenue column not provided - derived as quantity x unit_price.")

    # --- text cleaning -------------------------------------------------------
    if "review_text" in data.columns:
        data["review_text"] = data["review_text"].map(_clean_text)
    for text_col in ["product", "category"]:
        if text_col in data.columns:
            data[text_col] = data[text_col].astype(str).str.strip()
            data.loc[data[text_col].isin(["", "nan", "None"]), text_col] = np.nan

    # --- drop fully-empty and duplicate rows ---------------------------------
    data = data.dropna(how="all")
    dup = data.duplicated().sum()
    if dup:
        data = data.drop_duplicates()
        notes.append(f"Removed {dup} duplicate row(s).")

    # --- handle missing values -----------------------------------------------
    for num_col in ["quantity", "unit_price", "revenue", "rating"]:
        if num_col in data.columns:
            missing = int(data[num_col].isna().sum())
            if missing:
                fill = data[num_col].median()
                data[num_col] = data[num_col].fillna(fill)
```

```
                notes.append(f"Filled {missing} missing '{num_col}' value(s) with median
({fill:.2f}).")

    if "category" in data.columns:
        missing = int(data["category"].isna().sum())
        if missing:
            data["category"] = data["category"].fillna("Unknown")
            notes.append(f"Filled {missing} missing 'category' value(s) with 'Unknown'.")

    if "product" in data.columns:
        missing = int(data["product"].isna().sum())
        if missing:
            data["product"] = data["product"].fillna("Unknown product")

    # remove rows that have no usable revenue signal
    if "revenue" in data.columns:
        before = len(data)
        data = data[data["revenue"].notna()]
        removed = before - len(data)
        if removed:
            notes.append(f"Removed {removed} row(s) with no usable revenue value.")

    data = data.reset_index(drop=True)
    notes.insert(0, f"Started with {n_start} rows, {len(data)} rows after cleaning.")
    return data, notes


def aggregate_reviews(df: pd.DataFrame, positive_threshold: float = 4.0) -> pd.DataFrame
| None:
    """
    Aggregate review data to product level: average rating and positive
    review ratio (share of ratings >= positive_threshold).
    Returns None if no rating data is available.
    """
    if "rating" not in df.columns or "product" not in df.columns:
        return None

    grp = df.groupby("product").agg(
        avg_rating=("rating", "mean"),
        review_count=("rating", "count"),
    )
    pos = (
        df.assign(_pos=(df["rating"] >= positive_threshold).astype(int))
        .groupby("product")["_pos"]
        .mean()
        .rename("positive_review_ratio")
    )
    out = grp.join(pos).reset_index().sort_values("review_count", ascending=False)
    return out
```

8.3 src/analytics.py — Analytics Layer

```
"""
Analytics Layer
---------------
Descriptive / diagnostic analysis: computes the key performance indicators
(KPIs) and the exploratory-data-analysis (EDA) charts shown on the dashboard.
```

```
Charts are returned as Plotly figures, as specified in the methodology.
"""

from __future__ import annotations

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go


def compute_kpis(df: pd.DataFrame) -> dict:
    """Compute the headline KPIs for the dashboard."""
    kpis: dict = {}

    revenue = df["revenue"] if "revenue" in df.columns else pd.Series(dtype=float)
    kpis["total_revenue"] = float(revenue.sum()) if len(revenue) else 0.0

    if "order_id" in df.columns:
        kpis["total_orders"] = int(df["order_id"].nunique())
    else:
        kpis["total_orders"] = int(len(df))  # one row == one order

    kpis["total_units"] = int(df["quantity"].sum()) if "quantity" in df.columns else 0
    kpis["unique_products"] = int(df["product"].nunique()) if "product" in df.columns
else 0
    kpis["unique_categories"] = int(df["category"].nunique()) if "category" in df.columns
else 0
    kpis["unique_customers"] = int(df["customer_id"].nunique()) if "customer_id" in
df.columns else None

    kpis["avg_order_value"] = (
        kpis["total_revenue"] / kpis["total_orders"] if kpis["total_orders"] else 0.0
    )
    kpis["avg_rating"] = float(df["rating"].mean()) if "rating" in df.columns else None
    kpis["positive_review_ratio"] = (
        float((df["rating"] >= 4.0).mean()) if "rating" in df.columns else None
    )
    return kpis


def revenue_by_category(df: pd.DataFrame) -> go.Figure | None:
    if "category" not in df.columns or "revenue" not in df.columns:
        return None
    agg = (
        df.groupby("category")
["revenue"].sum().sort_values(ascending=False).reset_index()
    )
    fig = px.bar(
        agg, x="category", y="revenue", title="Revenue by Category",
        labels={"revenue": "Revenue", "category": "Category"}, color="revenue",
        color_continuous_scale="Blues",
    )
    fig.update_layout(coloraxis_showscale=False, xaxis_tickangle=-30)
    return fig


def top_products(df: pd.DataFrame, n: int = 10) -> go.Figure | None:
    if "product" not in df.columns or "revenue" not in df.columns:
```

```
        return None
    agg = (
        df.groupby("product")
["revenue"].sum().sort_values(ascending=False).head(n).reset_index()
    )
    fig = px.bar(
        agg, x="revenue", y="product", orientation="h",
        title=f"Top {n} Products by Revenue", color="revenue",
        color_continuous_scale="Tealgrn",
        labels={"revenue": "Revenue", "product": "Product"},
    )
    fig.update_layout(coloraxis_showscale=False, yaxis={"categoryorder": "total
ascending"})
    return fig


def rating_distribution(df: pd.DataFrame) -> go.Figure | None:
    if "rating" not in df.columns:
        return None
    fig = px.histogram(
        df, x="rating", nbins=10, title="Customer Rating Distribution",
        labels={"rating": "Rating"}, color_discrete_sequence=["#f59e0b"],
    )
    fig.update_layout(bargap=0.1, yaxis_title="Count")
    return fig


def revenue_trend(df: pd.DataFrame) -> go.Figure | None:
    if "order_date" not in df.columns or "revenue" not in df.columns:
        return None
    tmp = df.copy()
    tmp["order_date"] = pd.to_datetime(tmp["order_date"], errors="coerce")
    tmp = tmp.dropna(subset=["order_date"])
    if tmp.empty:
        return None
    trend = (
        tmp.set_index("order_date")
        .resample("ME")["revenue"]
        .sum()
        .reset_index()
    )
    fig = px.line(
        trend, x="order_date", y="revenue", markers=True,
        title="Monthly Revenue Trend",
        labels={"order_date": "Month", "revenue": "Revenue"},
    )
    fig.update_traces(line_color="#2563eb")
    return fig


def category_share(df: pd.DataFrame) -> go.Figure | None:
    if "category" not in df.columns or "revenue" not in df.columns:
        return None
    agg = df.groupby("category")["revenue"].sum().reset_index()
    fig = px.pie(
        agg, names="category", values="revenue", title="Revenue Share by Category",
hole=0.45,
    )
```

```
    fig.update_traces(textposition="inside", textinfo="percent+label")
    return fig
```

8.4 src/ml_model.py — Machine Learning Layer

```
"""
Machine Learning Layer
----------------------
Trains a Random Forest Regressor to predict revenue from sales attributes and
customer feedback features, and evaluates it with R2, MAE, MSE and RMSE -
exactly the metrics listed in the research methodology.
"""

from __future__ import annotations

import numpy as np
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
from sklearn.preprocessing import OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline


# Features the model is allowed to use (whichever are present in the data).
NUMERIC_CANDIDATES = ["quantity", "unit_price", "rating"]
CATEGORICAL_CANDIDATES = ["category"]
TARGET = "revenue"


def available_features(df: pd.DataFrame) -> tuple[list[str], list[str]]:
    numeric = [c for c in NUMERIC_CANDIDATES if c in df.columns]
    categorical = [c for c in CATEGORICAL_CANDIDATES if c in df.columns]
    return numeric, categorical


def can_train(df: pd.DataFrame) -> tuple[bool, str]:
    if TARGET not in df.columns:
        return False, "No revenue column available to predict."
    numeric, categorical = available_features(df)
    if not numeric and not categorical:
        return False, "No usable feature columns (need at least quantity, unit_price,
rating or category)."
    if len(df) < 20:
        return False, "Need at least 20 rows to train a meaningful model."
    return True, ""


def train_revenue_model(df: pd.DataFrame, test_size: float = 0.2, random_state: int = 42)
-> dict:
    """Train the Random Forest model and return model, metrics and diagnostics."""
    numeric, categorical = available_features(df)
    features = numeric + categorical

    work = df[features + [TARGET]].dropna()
```

```
    X = work[features]
    y = work[TARGET]

    pre = ColumnTransformer(
        transformers=[
            ("num", "passthrough", numeric),
            ("cat", OneHotEncoder(handle_unknown="ignore"), categorical),
        ]
    )
    model = Pipeline(
        steps=[
            ("pre", pre),
            ("rf", RandomForestRegressor(n_estimators=200, random_state=random_state,
n_jobs=-1)),
        ]
    )

    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=test_size, random_state=random_state
    )
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)

    mse = mean_squared_error(y_test, y_pred)
    metrics = {
        "r2": float(r2_score(y_test, y_pred)),
        "mae": float(mean_absolute_error(y_test, y_pred)),
        "mse": float(mse),
        "rmse": float(np.sqrt(mse)),
        "n_train": int(len(X_train)),
        "n_test": int(len(X_test)),
    }

    # Feature importances mapped back to readable names.
    rf = model.named_steps["rf"]
    ohe = model.named_steps["pre"].named_transformers_["cat"]
    cat_names = list(ohe.get_feature_names_out(categorical)) if categorical else []
    feat_names = numeric + cat_names
    importances = pd.DataFrame(
        {"feature": feat_names, "importance": rf.feature_importances_}
    ).sort_values("importance", ascending=False)

    return {
        "model": model,
        "metrics": metrics,
        "features": features,
        "numeric": numeric,
        "categorical": categorical,
        "importances": importances,
        "y_test": y_test.reset_index(drop=True),
        "y_pred": pd.Series(y_pred, name="predicted"),
    }


def actual_vs_predicted_fig(y_test: pd.Series, y_pred: pd.Series) -> go.Figure:
    fig = px.scatter(
        x=y_test, y=y_pred,
        labels={"x": "Actual revenue", "y": "Predicted revenue"},
```

```
        title="Actual vs. Predicted Revenue (test set)",
        opacity=0.6,
    )
    lo = float(min(y_test.min(), y_pred.min()))
    hi = float(max(y_test.max(), y_pred.max()))
    fig.add_trace(
        go.Scatter(x=[lo, hi], y=[lo, hi], mode="lines",
                   line=dict(color="red", dash="dash"), name="Perfect prediction")
    )
    return fig


def importance_fig(importances: pd.DataFrame) -> go.Figure:
    top = importances.head(12).sort_values("importance")
    fig = px.bar(
        top, x="importance", y="feature", orientation="h",
        title="Feature Importance (Random Forest)",
        color="importance", color_continuous_scale="Purples",
    )
    fig.update_layout(coloraxis_showscale=False)
    return fig
```

8.5 src/recommendation.py — Recommendation Layer

```
"""
Recommendation Layer
--------------------
Rule-based logic that suggests the most suitable e-commerce business model -
B2B, B2C or D2C - based on analytical outcomes such as average order value,
sales volume, product diversity and customer satisfaction.

The logic is transparent and explainable: each model gets a score built from
interpretable business signals, and the reasoning is returned alongside the
recommendation so a non-technical user can understand *why*.
"""

from __future__ import annotations

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go


MODEL_INFO = {
    "B2B": {
        "name": "Business-to-Business (B2B)",
        "blurb": "Sell in bulk to other businesses. Fewer, larger orders with high "
                 "average order value and wholesale-style quantities.",
    },
    "B2C": {
        "name": "Business-to-Consumer (B2C)",
        "blurb": "Sell a broad catalogue to many individual consumers. High order "
                 "volume, wide product diversity and moderate order values.",
    },
    "D2C": {
        "name": "Direct-to-Consumer (D2C)",
        "blurb": "Sell your own focused brand directly to consumers. Narrow product "
                 "range, strong ratings and brand loyalty.",
```

```
    },
}


def compute_business_signals(df: pd.DataFrame, kpis: dict) -> dict:
    """Derive the business signals used by the recommendation rules."""
    total_orders = kpis.get("total_orders") or len(df)
    avg_order_value = kpis.get("avg_order_value", 0.0)
    avg_qty = float(df["quantity"].mean()) if "quantity" in df.columns else 0.0
    unique_products = kpis.get("unique_products", 0)
    unique_categories = kpis.get("unique_categories", 0)
    avg_rating = kpis.get("avg_rating")

    # Repeat-purchase signal (useful for D2C brand loyalty) if customer ids exist.
    repeat_ratio = None
    if "customer_id" in df.columns and df["customer_id"].nunique() > 0:
        counts = df.groupby("customer_id").size()
        repeat_ratio = float((counts > 1).mean())

    return {
        "avg_order_value": avg_order_value,
        "avg_quantity_per_order": avg_qty,
        "total_orders": total_orders,
        "unique_products": unique_products,
        "unique_categories": unique_categories,
        "avg_rating": avg_rating,
        "repeat_purchase_ratio": repeat_ratio,
    }


def recommend_business_model(signals: dict) -> dict:
    """
    Score each business model from the signals and return the recommendation,
    the full score breakdown and human-readable reasoning.
    Scores are 0-100 and normalised so the three models can be compared.
    """
    aov = signals["avg_order_value"]
    avg_qty = signals["avg_quantity_per_order"]
    n_products = signals["unique_products"]
    n_categories = signals["unique_categories"]
    rating = signals["avg_rating"] or 0.0
    repeat = signals["repeat_purchase_ratio"]

    scores = {"B2B": 0.0, "B2C": 0.0, "D2C": 0.0}
    reasons = {"B2B": [], "B2C": [], "D2C": []}

    # ---- B2B: bulk, high order value, large quantities ----
    if avg_qty >= 20:
        scores["B2B"] += 35; reasons["B2B"].append(f"Large average quantity per order
({avg_qty:.0f} units) suggests bulk/wholesale buying.")
    elif avg_qty >= 8:
        scores["B2B"] += 18; reasons["B2B"].append(f"Moderately high quantity per order
({avg_qty:.0f} units).")
    if aov >= 5000:
        scores["B2B"] += 35; reasons["B2B"].append(f"Very high average order value
({aov:,.0f}) is typical of business buyers.")
    elif aov >= 1500:
        scores["B2B"] += 18; reasons["B2B"].append(f"Above-average order value
```

```
({aov:,.0f}).")
    if n_categories <= 4:
        scores["B2B"] += 10; reasons["B2B"].append("Narrow category range fits a
specialised supplier.")

    # ---- B2C: high volume, broad catalogue, moderate AOV ----
    if signals["total_orders"] >= 500:
        scores["B2C"] += 25; reasons["B2C"].append(f"High order volume
({signals['total_orders']:,} orders) points to a mass consumer market.")
    elif signals["total_orders"] >= 150:
        scores["B2C"] += 14; reasons["B2C"].append(f"Healthy order volume
({signals['total_orders']:,} orders).")
    if n_products >= 50:
        scores["B2C"] += 25; reasons["B2C"].append(f"Wide product range ({n_products}
products) suits a broad consumer catalogue.")
    elif n_products >= 15:
        scores["B2C"] += 14; reasons["B2C"].append(f"Diverse product range ({n_products}
products).")
    if n_categories >= 5:
        scores["B2C"] += 18; reasons["B2C"].append(f"Many categories ({n_categories})
indicate a multi-segment marketplace.")
    if 1 <= avg_qty < 8:
        scores["B2C"] += 15; reasons["B2C"].append(f"Small basket size ({avg_qty:.1f}
units/order) is typical of individual shoppers.")
    if 100 <= aov < 1500:
        scores["B2C"] += 12; reasons["B2C"].append(f"Moderate order value ({aov:,.0f})
fits retail consumers.")

    # ---- D2C: focused brand, strong ratings, repeat buyers ----
    if n_categories <= 3:
        scores["D2C"] += 22; reasons["D2C"].append(f"Focused catalogue ({n_categories}
categor{'y' if n_categories==1 else 'ies'}) fits a single-brand D2C strategy.")
    if n_products <= 25:
        scores["D2C"] += 16; reasons["D2C"].append(f"Tight product line ({n_products}
products) suits a direct brand.")
    if rating >= 4.3:
        scores["D2C"] += 30; reasons["D2C"].append(f"Excellent average rating
({rating:.2f}) signals strong brand loyalty.")
    elif rating >= 4.0:
        scores["D2C"] += 18; reasons["D2C"].append(f"Strong average rating
({rating:.2f}).")
    if repeat is not None and repeat >= 0.25:
        scores["D2C"] += 20; reasons["D2C"].append(f"High repeat-purchase ratio
({repeat:.0%}) shows loyal direct customers.")
    if 50 <= aov < 1500:
        scores["D2C"] += 8; reasons["D2C"].append("Consumer-level order value consistent
with direct selling.")

    # Normalise to 0-100 for comparison.
    max_score = max(scores.values()) or 1.0
    norm = {m: round(100 * s / max_score, 1) for m, s in scores.items()}

    recommended = max(norm, key=norm.get)
    # Confidence = gap between best and second best.
    ranked = sorted(norm.values(), reverse=True)
    gap = ranked[0] - ranked[1] if len(ranked) > 1 else 100
    confidence = "High" if gap >= 25 else "Medium" if gap >= 10 else "Low"
```

```
    return {
        "recommended": recommended,
        "scores": norm,
        "raw_scores": scores,
        "reasons": reasons,
        "confidence": confidence,
    }


def score_fig(scores: dict) -> go.Figure:
    data = pd.DataFrame({"model": list(scores.keys()), "score": list(scores.values())})
    colors = {"B2B": "#6366f1", "B2C": "#10b981", "D2C": "#f59e0b"}
    fig = px.bar(
        data, x="model", y="score", title="Business Model Suitability Score",
        color="model", color_discrete_map=colors, text="score",
        labels={"model": "Business Model", "score": "Suitability (0-100)"},
    )
    fig.update_traces(textposition="outside")
    fig.update_layout(showlegend=False, yaxis_range=[0, 110])
    return fig
```